Ministério da Educação
Universidade Tecnológica Federal do Paraná
Câmpus Santa Helena
Bacharelado em Ciência da Computação

Disciplina:	Projeto e Desenvolvimento de Sistemas Web		
Aluno:	Gustavo Galafaci Conte	R.A.:	2576635
Aluno:	Igor Carvalho Marchi	R.A.:	2677229
Aluno:	Jefferson korte junior	R.A.:	2651114

ATIVIDADE

Sobre o projeto

A TecShop é uma loja online fictícia que o grupo desenvolveu para o primeiro trabalho da disciplina. A ideia foi fazer um e-commerce, porque dá para encaixar quase tudo o que foi visto em aula sem precisar inventar: a lista de produtos usa consumo de API, o carrinho usa estado, o checkout usa formulário com validação e a navegação entre as páginas usa as rotas do Next.js.

O site tem duas partes: a loja, que é o que o cliente vê, e um painel administrativo separado, que é uma área restrita onde só contas de administrador entram. No painel dá para cadastrar produtos, acompanhar pedidos e olhar os relatórios.

Ainda não existe back-end. Os produtos são buscados uma vez numa API pública chamada Fake Store API e, a partir daí, ficam salvos no localStorage do navegador. Fizemos assim porque a API é só de leitura: sem isso, o administrador não conseguiria editar nem cadastrar nada. Como a próxima etapa da disciplina é o back-end, o front-end já foi organizado para trocar o localStorage por chamadas ao servidor sem refazer as telas.

Tecnologias usadas

Tecnologia	Para que usamos
Next.js 16 (App Router)	Rotas por pastas, layouts, navegação sem recarregar a página e o proxy.js (antigo middleware) que protege as rotas privadas.
React 19	Componentes, props, estado (useState), efeitos (useEffect) e Context API.
Tailwind CSS 4	Estilo utility-first, layout responsivo (mobile-first) e modo escuro com a classe dark.
React Hook Form + Zod	Formulários de login, cadastro, checkout, avaliação e produto, com as regras de validação num schema.
Fake Store API	Carga inicial dos produtos com fetch, tratando carregamento e erro.
localStorage + cookie	Guardam os dados enquanto não há servidor; o cookie de sessão é lido pelo proxy.js.

Atendimento ao escopo mínimo

A tabela abaixo mostra onde cada requisito obrigatório do trabalho foi atendido no código.

Requisito	Como foi atendido	Onde está
Homepage	Página de entrada com banner, promoções, filtro por categoria e busca. A barra de navegação (Navbar) aparece em todas as páginas da loja e leva a todas as áreas.	app/page.jsx components/Navbar.jsx

Páginas públicas (mínimo 2)	Seis páginas abertas a qualquer visitante: lista de produtos, detalhe do produto, sobre, feedback, carrinho e cadastro.	app/produtos app/produtos/[id] app/sobre, app/feedback app/carrinho, app/cadastro
Área privada	Área do cliente (checkout, meus pedidos e lista de desejos) para qualquer usuário logado, e painel administrativo só para ADMIN.	app/(privado)/ app/admin/
Sistema de login	Tela de login com React Hook Form + Zod. Quando dá certo, grava a sessão e devolve a pessoa para a página que ela queria abrir.	app/login/page.jsx contexto/UsuarioContext.jsx
Controle de acesso (middleware)	Duas camadas: o proxy.js verifica o cookie de sessão antes de a página abrir e redireciona para /login; a RotaProtegida confere de novo no navegador e bloqueia cliente no painel.	proxy.js components/RotaProtegida.jsx app/(privado)/layout.jsx app/admin/layout.jsx

Como o projeto está organizado

- Dentro de **app/** estão as 21 páginas (cada pasta vira uma rota) e os 3 layouts: o geral, o da área do cliente e o do painel.
- Dentro de **app/(privado)/** ficam as páginas da área do cliente (checkout, meus pedidos e lista de desejos). Os parênteses fazem o Next ignorar o nome da pasta na URL, então app/(privado)/checkout continua sendo /checkout. Assim todas ficam protegidas por um único layout.
- Dentro de **components/** existem 19 componentes reaproveitados.
- Dentro de **contexto/** existem os 9 contextos, que guardam os dados compartilhados.

- Dentro de **dados/** ficam os dados fixos e as contas de preço.
- O arquivo **proxy.js**, na raiz, faz o controle de acesso das rotas privadas.

As páginas que começam com /admin são do painel. As outras são da loja.

Componentização

Procuramos não repetir tela. Os componentes recebem os dados por props e avisam o pai por funções (os dados descem, os eventos sobem):

- CardProduto recebe o produto por props e é usado na home, na lista de produtos e na lista de desejos.
- FormProduto é o mesmo formulário para cadastrar e para editar: recebe o produto (quando é edição) e a função aoSalvar.
- FormFeedback recebe aoEnviar, por isso serve tanto para avaliar um produto quanto para avaliar o site.
- Sidebar e BarraBusca não guardam estado: recebem o valor e a função que muda o valor, e quem guarda o estado é a página.
- GraficoBarras desenha os gráficos do painel só com CSS, sem biblioteca.
- RotaProtegida envolve qualquer área privada; com a prop apenasAdmin também exige conta de administrador.

Os hooks

Essa é a parte que mais usamos. Vai um resumo de cada um e onde ele aparece.

useState

É o hook mais usado do projeto. Serve para guardar tudo o que muda na tela: o texto digitado na busca, a categoria selecionada, se a página ainda está carregando. Exemplo da busca de produtos:

```
const [busca, setBusca] = useState("");

const produtosFiltrados = produtosAtivos.filter((p) =>
   p.nome.toLowerCase().includes(busca.toLowerCase())
);
```

Toda vez que a pessoa digita uma letra, o setBusca atualiza o estado, a página é desenhada de novo e a lista já aparece filtrada.

useEffect

Usamos o useEffect para três coisas.

Buscar dados quando a página abre. O array vazio no final faz ele rodar só uma vez. A resposta é conferida com resposta.ok, porque o fetch não dá erro sozinho quando a API devolve 404 ou 500:

```
useEffect(() => {
        async function buscarNaApi() {
        try {
        const resposta = await fetch("https://fakestoreapi.com/products");
        if (!resposta.ok) {
            throw new Error("A API respondeu com erro " + resposta.status);
        }
        const dados = await resposta.json();
         setProdutos(dados.map(converterDaApi));
        } catch (e) {
         setErro(e.message);
        } finally {
         setCarregando(false);
        }
        }
   buscarNaApi();
}, []);
```

Com isso a tela trata os estados de carregando, erro, lista vazia ("Nenhum produto encontrado") e sucesso.

Reagir quando alguma coisa muda. Na página de detalhe do produto, o params.id está no array de dependências, então toda vez que a URL muda o efeito roda de novo. O mesmo acontece na RotaProtegida, que observa carregou e usuario para decidir se manda a pessoa para o login.

Salvar no localStorage. Em todos os contextos existe um efeito que grava os dados sempre que eles mudam:

```
useEffect(() => {
        if (carregou) {
      localStorage.setItem("carrinho", JSON.stringify(itens));
        }
}, [itens, carregou]);
```

Aquele carregou foi um problema que levamos um tempo para entender. Sem ele, assim que a página abria o efeito salvava o carrinho vazio por cima do que estava guardado, e tudo sumia. O carregou só vira verdadeiro depois que terminamos de ler o localStorage, aí o salvamento acontece de verdade.

Outro detalhe: a leitura do localStorage precisa estar dentro do useEffect. O Next renderiza uma parte da página no servidor, e lá o localStorage não existe. Se tentar ler fora do efeito, dá erro.

Hooks do Next

- **useParams** pega o que veio na URL. Em /produtos/3, o params.id vale "3". Vem como texto, por isso usamos Number() antes de comparar com o id do produto.

- **useRouter** manda a pessoa para outra página pelo código. Depois do login usamos router.replace, e não push, para o botão "voltar" do navegador não cair de novo na tela de login.
- **usePathname** diz qual página está aberta. Usamos para marcar o item do menu, esconder a navbar da loja dentro do painel e para a RotaProtegida saber para onde voltar depois do login.

useForm (React Hook Form + Zod)

Os formulários de cadastro, login, checkout, avaliação e cadastro de produto usam React Hook Form junto com Zod. O Zod é onde ficam as regras, que acabam ficando bem fáceis de ler:

```
const schema = z.object({
        nome: z.string().min(3, "O nome precisa ter pelo menos 3 letras"),
        email: z.string().email("Digite um e-mail válido"),
        senha: z.string().min(6, "A senha precisa ter pelo menos 6 caracteres"),
});
```

No cadastro precisamos conferir se as duas senhas digitadas eram iguais. Isso não dá para fazer campo por campo, porque depende de dois campos ao mesmo tempo. Para isso serve o refine, que olha o formulário inteiro. O mesmo recurso é usado no cadastro de produto, para impedir que o preço promocional seja maior que o preço normal.

useContext

Esse resolveu o maior problema que tivemos. O carrinho precisa ser lido pela navbar, pela página de produtos, pela do carrinho e pela de checkout. Passar isso de componente em componente por props ficaria impossível.

Com o Context, criamos o dado em um lugar só e qualquer página consegue ler. São 9 contextos no projeto:

Contexto	**O que guarda**
Tema	se o site está no modo claro ou escuro
Usuário	quem está logado, a lista de cadastrados e o cookie de sessão
Produtos	os produtos da loja
Categorias	as categorias
Promoções	as promoções
Carrinho	o carrinho de compras
Lista de desejos	a lista de desejos de cada cliente
Pedidos	todos os pedidos
Feedback	as avaliações dos produtos e do site

Para não ter que importar o useContext em toda página, cada contexto exporta um atalho:

```
export function useCarrinho() {
        return useContext(CarrinhoContext);
}
```

Aí na página é só escrever const { itens, total } = useCarrinho().

Uma coisa que descobrimos na prática: a ordem dos providers no layout.jsx importa. O contexto de pedidos precisa saber quem está logado, então ele tem que ficar por dentro do contexto de usuário, nunca por fora.

Mapa de rotas

Rota	Acesso	O que mostra
/	Pública	Página inicial com promoções, categorias e busca
/produtos	Pública	Catálogo completo com busca por nome
/produtos/[id]	Pública	Detalhe do produto, avaliações e lista de desejos (rota dinâmica)
/sobre	Pública	Página institucional da empresa
/feedback	Pública	Avaliações do site (para escrever, precisa estar logado)
/carrinho	Pública	Carrinho com quantidade e total
/login e /cadastro	Pública	Entrar e criar conta
/checkout	Cliente logado	Formulário de entrega e pagamento
/meus-pedidos	Cliente logado	Histórico e status dos pedidos da pessoa
/lista-desejos	Cliente logado	Produtos guardados para depois
/admin	Só ADMIN	Dashboard com números e gráficos
/admin/produtos /admin/produtos/novo /admin/produtos/[id]	Só ADMIN	Listar, cadastrar, editar, ativar e excluir produtos
/admin/categorias	Só ADMIN	Gerenciar categorias
/admin/pedidos /admin/pedidos/[numero]	Só ADMIN	Acompanhar pedidos e mudar o status
/admin/avaliacoes	Só ADMIN	Moderar avaliações
/admin/promocoes	Só ADMIN	Criar promoções por produto ou categoria
/admin/relatorios	Só ADMIN	Relatórios por período, categoria e produto

Autenticação e controle de acesso

Tipos de usuário

O sistema tem dois tipos de usuário, CLIENTE e ADMIN. Quem se cadastra pelo site sempre vira CLIENTE. A conta de administrador já vem criada junto com o sistema, porque senão ninguém conseguiria entrar no painel na primeira vez:

- E-mail: admin@tecshop.com
- Senha: admin123

O que cada um pode fazer:

Ação	Visitante	Cliente	Admin
Ver produtos e buscar	Sim	Sim	Sim
Mexer no carrinho	Sim	Sim	Sim
Finalizar a compra	Não	Sim	Sim
Lista de desejos	Não	Sim	Sim
Ver os próprios pedidos	Não	Sim	Sim
Avaliar produto ou o site	Não	Sim	Sim
Entrar no painel	Não	Não	Sim

Como o login funciona

A tela de login valida os campos com Zod e chama a função entrar do contexto de usuário. Se o e-mail e a senha conferem e a conta está ativa, o contexto guarda o usuário no localStorage e grava um cookie de sessão (tecshop_sessao). Esse

cookie existe porque o proxy.js roda antes da página e não consegue ler o localStorage, só cookies. Ao sair, o cookie é apagado.

Proteção das rotas: duas camadas

Camada 1: proxy.js (middleware). No Next.js 16 o antigo middleware.js passou a se chamar proxy.js. Ele roda antes de a rota ser entregue, inclusive nas navegações feitas pelo <Link>. Se não houver cookie de sessão, a rota é bloqueada e a pessoa vai para o login, levando junto o endereço que tentou abrir:

```
export function proxy(request) {
        const sessao = request.cookies.get("tecshop_sessao")?.value;

        if (!sessao) {
        const urlLogin = new URL("/login", request.url);
      urlLogin.searchParams.set("redirect", request.nextUrl.pathname);
        return NextResponse.redirect(urlLogin);
        }

        return NextResponse.next();
}

export const config = {
        matcher: ["/admin/:path*", "/checkout", "/meus-pedidos", "/lista-desejos"],
};
```

Camada 2: RotaProtegida (no navegador). É usada no layout do grupo (privado) e no layout do painel. Primeiro ela espera o contexto terminar de ler o localStorage (carregou), para não expulsar quem só deu F5. Depois decide: sem login, redireciona para /login?redirect=...; logado como cliente dentro do painel, mostra "Acesso negado"; caso contrário, mostra a página.

```
useEffect(() => {
        if (carregou && !usuario) {
      router.replace(`/login?redirect=${encodeURIComponent(caminho)}`);
        }
}, [carregou, usuario, caminho, router]);
```

```
if (!carregou || !usuario) {
        return <Carregando texto="Verificando acesso..." />;
}

if (apenasAdmin && usuario.tipo !== "ADMIN") {
        return <AcessoNegado />;
}

return children;
```

Como os layouts envolvem todas as páginas de dentro, basta a proteção estar neles. Isso vale inclusive para quem digita /admin ou /meus-pedidos direto na barra de endereço.

O fluxo completo

1. O visitante tenta abrir /meus-pedidos sem estar logado.
2. O proxy.js não encontra o cookie e redireciona para /login?redirect=/meus-pedidos.
3. A pessoa faz login. O contexto grava o usuário e o cookie de sessão.
4. A tela de login lê o redirect e chama router.replace("/meus-pedidos"). Só aceitamos caminhos internos (começando com uma barra), para ninguém montar um link que leve a outro site depois do login.
5. Agora o cookie existe, o proxy.js libera e a RotaProtegida mostra a página.

Quem já está logado e abre o login

Se a pessoa já está logada e abre /login ou /cadastro, o formulário não aparece de novo. No lugar dele aparece a mensagem "Você já está logado", com o nome de quem entrou, e a pergunta "Tem certeza que deseja sair?". O botão "Sim, sair" chama a função sair, que apaga o usuário e o cookie de sessão; o botão "Não,

voltar para a loja" volta para a página inicial sem mudar nada. O botão "Sair" do menu usa essa mesma tela como confirmação.

Usamos o estado acabouDeEntrar para essa tela não piscar logo depois de um login que deu certo, no instante entre o login e a mudança de página.

Cada cliente só vê o que é seu

Os pedidos ficam todos numa lista só, com o e-mail do cliente anotado em cada um, e a página "Meus pedidos" filtra por esse e-mail. O painel mostra a lista inteira. A lista de desejos funciona parecido, só que guardada num objeto em que a chave é o e-mail da pessoa.

Também tivemos cuidado para o painel nunca mostrar a senha de ninguém: a função que devolve a lista de clientes monta objetos novos sem o campo da senha, em vez de devolver o usuário inteiro.

O que foi percebido que está limitado

Preferimos deixar isso escrito em vez de fingir que não existe. Quase tudo aqui se resolve com o back-end da próxima etapa.

- **As senhas ficam salvas sem criptografia.** Como não tem servidor, elas vão para o localStorage em texto puro. Num sistema de verdade a senha nunca pode ser guardada assim: o certo é guardar um hash dela num banco de dados no servidor.
- **A proteção ainda depende do navegador.** O proxy.js e a RotaProtegida bloqueiam e redirecionam corretamente, mas tanto o cookie quanto o localStorage podem ser alterados pelo F12 por quem está usando. A checagem de permissão de verdade tem que acontecer no servidor, com um token assinado.

- **Os dados são do navegador, não da loja.** Se abrir o site em outro computador, os produtos voltam a ser buscados na API e começa tudo do zero.
- **O tema pisca claro por um instante** ao carregar quando o modo escuro está selecionado, porque o tema só é lido depois que a página aparece. Daria para arrumar com um script no cabeçalho do HTML, mas achamos que não valia a complicação.

Como rodar e testar

npm install
npm run dev

Depois é só abrir http://localhost:3000. Na primeira vez o site precisa de internet, porque é quando ele busca os produtos na API. Depois disso eles ficam salvos.

Para Inciar do zero, basta apagar os dados do site pelo F12, na aba Application, em Local storage e Cookies.